State-Space Construction for Binary Session Types

Alexandre Zua Caldeira
Independent Researcher
zua@bica-tools.org

March 2026

Abstract

Session types describe the communication protocols that govern interactions with objects. Given a session type S, one can construct a *state space* $\mathcal{L}(S)$: a labelled transition system whose states represent protocol stages and whose transitions represent method calls. In this paper we give a precise inductive definition of $\mathcal{L}(S)$ for binary session types—comprising termination (**end**), branch ($\&\{\cdots\}$), selection ($\oplus\{\cdots\}$), recursion ($\mu X.\, S$), and parallel composition ($S_1 \parallel S_2$)—and prove that $\mathcal{L}(S)$ is a *finite bounded poset* under the reachability ordering. Recursion introduces cycles, handled by quotienting over strongly connected components; parallel composition produces a product state space whose bounded poset structure is inherited componentwise. We establish determinism and compositionality, validate the construction on 79 benchmark protocols, and mechanise the key lemmas in Lean 4. This result is the foundation for showing that session-type state spaces possess full lattice structure.

Keywords: session types, state spaces, bounded posets, lattices, recursion, parallel composition, reachability ordering, structural induction

1 Introduction

Session types [14, 22] describe the sequences of operations that a client may perform on an object or a communication channel. They serve as *behavioural contracts*: the type prescribes which methods are available at each protocol stage, and a well-typed client is guaranteed to follow the protocol. Since their introduction by Honda [14] for the π-calculus, session types have found application in object-oriented languages [12, 22, 13], multiparty protocols [15], and tool support [17, 24].

A session type naturally induces a *state space*: a labelled transition system (LTS) whose states are protocol stages and whose transitions correspond to method invocations. This state space is implicit in every session-type system—it is how clients understand "what can I do next?"—yet it is rarely treated as a first-class mathematical object. When it is, the focus is typically on the transition structure (for type checking or monitoring) rather than on the *order-theoretic* properties of the state space.

Observation. The state spaces of session types have remarkably regular structure. They are always finite (for finite types), always acyclic (again, for finite types), and always have a unique initial state and a unique terminal state. In order-theoretic language, they form *bounded posets*—partially ordered sets with a greatest element (the initial protocol state) and a least element (the terminated state).

This observation is the starting point for a deeper investigation. We conjecture—and verify empirically on 79 benchmarks and 478,000+ exhaustively enumerated types—that session-type state spaces are not merely bounded posets but full *lattices*: every pair of states has a meet and

a join. The parallel composition constructor $\parallel$ *forces* lattice structure by producing a product lattice (Section 3.6 and Theorem 5.3). The full lattice proof is the subject of forthcoming work; this paper establishes the construction and the bounded poset property on which it rests.

This paper. We define the state-space construction for binary session types covering all six constructors: termination (**end**), branch ($\&\{m_1{:}S_1, \ldots\}$), selection ($\oplus\{l_1{:}S_1, \ldots\}$), recursion ($\mu X.\, S$), parallel composition ($S_1 \parallel S_2$), and variables (X). We:

1. Define $\mathcal{L}(S)$ by structural induction (Section 3), including the placeholder-merge technique for recursion and the product construction for parallel.
2. Define the reachability ordering and prove it is a partial order on acyclic state spaces (Section 4); for recursive types, the SCC quotient recovers a partial order.
3. Prove that $\mathcal{L}(S)$ is a *finite bounded poset* (Section 5), with explicit cases for recursion (Top Absorption) and parallel (product of bounded posets).
4. Establish determinism and compositionality (Section 6).
5. Validate the construction on 79 benchmark protocols and mechanise the key lemmas in Lean 4 (Section 7).

Outline. Section 2 defines the session type grammar and well-formedness. Section 3 gives the state-space construction for all six constructors, including recursion (Rule 5, with the Top Absorption Lemma) and parallel (Rule 6, product construction). Section 4 defines the reachability ordering and proves it is a partial order. Section 5 proves the bounded poset theorem. Section 6 establishes determinism and compositionality. Section 7 presents worked examples including recursion and parallel. Section 8 discusses the lattice conjecture and related work. Section 9 concludes.

2 Session Types

A *session type* describes the protocol that a client must follow when interacting with a single object.

Definition 2.1 (Session type)**.** The set of *session types* is generated by the grammar:

$$S \;::=\; \mathbf{end} \;\mid\; \&\{m_1{:}S_1,\ \ldots,\ m_n{:}S_n\} \;\mid\; \oplus\{l_1{:}S_1,\ \ldots,\ l_n{:}S_n\} \;\mid\; \mu X.\, S \;\mid\; X \;\mid\; S_1 \parallel S_2$$

where $n \geq 1$, the labels $m_1, \ldots, m_n$ (respectively $l_1, \ldots, l_n$) are pairwise distinct, each S_i is a session type, X ranges over a countable set of type variables, and $\mu X.\, S$ binds X in S.

The six constructors are:

- **end** (termination): the protocol is complete; no further operations are available.
- $\&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$ (branch / external choice): the *client* chooses one of the methods m_i and the protocol continues as S_i.
- $\oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}$ (selection / internal choice): the *object* chooses one of the labels l_i (typically representing a return value or status code) and the protocol continues as S_i.
- $\mu X.\, S$ (recursion): binds the variable X in S; unfolding replaces X with the entire $\mu X.\, S$.
- X (variable): a recursion point; must occur under a binding μX.
- $S_1 \parallel S_2$ (parallel): two sub-protocols execute concurrently on the same object, synchronising at termination.

Definition 2.2 (Well-formedness)**.** A session type is *well-formed* if: (i) every variable X occurs under a binding $\mu X.\, \cdot$, (ii) every recursion is *guarded*: the body of $\mu X.\, S$ performs at least one action (branch or selection) before reaching X, (iii) the sub-types of $\parallel$ are terminating (every execution path reaches **end**), and (iv) in $S_1 \parallel S_2$, the label sets of S_1 and S_2 are disjoint (no method name appears in both branches). Session types are identified up to alpha-renaming of bound variables (i.e., $\mu X.\, S$ and $\mu Y.\, S[Y/X]$ are the same type).

Definition 2.3 (Size)**.** The *syntactic size* of a session type is defined inductively:

$$|\mathbf{end}| = 1 \qquad |X| = 1$$
$$|\&\{m_1{:}S_1, \ldots, m_n{:}S_n\}| = 1 + \textstyle\sum_{i=1}^{n} |S_i| \qquad |\mu X.\, S| = 1 + |S|$$
$$|\oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}| = 1 + \textstyle\sum_{i=1}^{n} |S_i| \qquad |S_1 \parallel S_2| = 1 + |S_1| + |S_2|$$

This counts AST nodes and serves as the measure for structural induction. The *state count* $|Q^S|$ differs: for parallel, $|Q^{S_1 \| S_2}| = |Q^{S_1}| \cdot |Q^{S_2}|$ (product), which can be exponentially larger than $|S|$.

3 State-Space Construction

We now define the state-space construction $\mathcal{L}(S)$, which maps a well-formed session type to a labelled transition system.

Definition 3.1 (State space)**.** A *state space* is a tuple $\mathcal{L} = (Q,\ \Sigma,\ \delta,\ q_\top,\ q_\bot)$ where:

- Q is a finite set of *states* (protocol stages),
- Σ is a finite *alphabet* of labels (method names),
- $\delta \subseteq Q \times \Sigma \times Q$ is the *transition relation*,
- $q_\top \in Q$ is the *initial* (top) state,
- $q_\bot \in Q$ is the *terminal* (bottom) state.

Notation 3.2. We write $s \xrightarrow{m} s'$ for $(s, m, s') \in \delta$, and δ^S, $q_\top^S$, $q_\bot^S$ for the components of $\mathcal{L}(S)$. We write $Pos(S) = (Q^S, \to^*)$ for the induced reachability poset (or its SCC quotient for recursive types; see Definition 5.2).

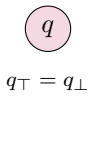
The construction is defined by structural induction on S. We assume an inexhaustible supply of fresh state identifiers; in each case, the states introduced are distinct from all states introduced in sub-constructions.

3.1 Rule 1: Termination

Definition 3.3 (Construction for **end**)**.**

$$\mathcal{L}(\mathbf{end}) = (\{q\},\ \emptyset,\ \emptyset,\ q,\ q)$$

A single state that serves as both initial and terminal state, with no transitions.



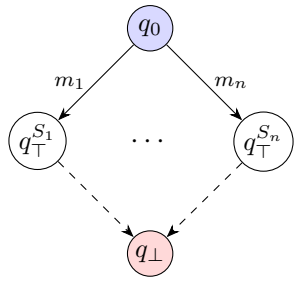
3.2 Rule 2: Branch (External Choice)

Definition 3.4 (Construction for branch)**.** Given $S = \&\{m_1{:}S_1,\ \ldots,\ m_n{:}S_n\}$, let $\mathcal{L}(S_i) = (Q_i, \Sigma_i, \delta_i, q_\top^{S_i}, q_\bot^{S_i})$ for each i, with the Q_i pairwise disjoint. Introduce a fresh state $q_0 \notin \bigcup_i Q_i$ and a shared terminal state $q_\bot$. Then:

$$\mathcal{L}(S) = \Big(\{q_0\} \cup \textstyle\bigcup_{i=1}^{n} Q_i[\,q_\bot^{S_i} \mapsto q_\bot],\ \{m_1, \ldots, m_n\} \cup \bigcup_i \Sigma_i,\ \delta,\ q_0,\ q_\bot\Big)$$

where $Q_i[\,q_\bot^{S_i} \mapsto q_\bot]$ denotes the state set of the i-th sub-construction with its terminal state identified with the shared $q_\bot$, and the transition relation is:

$$\delta = \{(q_0, m_i, q_\top^{S_i}) \mid 1 \le i \le n\} \ \cup\ \textstyle\bigcup_{i=1}^{n} \delta_i[\,q_\bot^{S_i} \mapsto q_\bot]$$



The fresh state q_0 becomes the initial state with outgoing transitions labelled $m_1, \ldots, m_n$ leading into each sub-construction. All branches share a common terminal state $q_\bot$, obtained by identifying the terminal states of the sub-constructions.

3.3 Rule 3: Selection (Internal Choice)

Definition 3.5 (Construction for selection)**.** Given $S = \oplus\{l_1{:}S_1,\ \ldots,\ l_n{:}S_n\}$, the construction is structurally identical to the branch case: introduce a fresh q_0, share the terminal state, and create transitions $(q_0, l_i, q_\top^{S_i})$. The only difference is that the transitions from q_0 are *selection transitions*: the choice is made by the object, not the client.

$$\mathcal{L}(S) = \Big(\{q_0\} \cup \textstyle\bigcup_{i=1}^{n} Q_i[\,q_\bot^{S_i} \mapsto q_\bot],\ \{l_1, \ldots, l_n\} \cup \bigcup_i \Sigma_i,\ \delta,\ q_0,\ q_\bot\Big)$$

where δ is defined as for branch. We annotate the transitions from q_0 as selection transitions, written $q_0 \xrightarrow{l_i}_{\oplus} q_\top^{S_i}$, to distinguish them from branch transitions.

Remark 3.6 (Branch vs. selection)*.* The state-space *structure* (graph topology) of branch and selection is identical. The distinction lies in the *ownership* of the choice: in a branch, the client decides which transition to take; in a selection, the object decides. This distinction is irrelevant for the order-theoretic properties proved in this paper (bounded poset, acyclicity) but becomes critical for subtyping and conformance checking.

3.4 Rule 4: Variable

Definition 3.7 (Construction for variable)**.** When the construction encounters a variable X during the recursive building of $\mathcal{L}(\mu X.\, S')$, it returns the placeholder state q_X previously allocated for X. No new states or transitions are introduced. The construction is undefined for free variables; this case is excluded by well-formedness (Definition 2.2).

3.5 Rule 5: Recursion

Definition 3.8 (Construction for recursion)**.** Given $S = \mu X.\, S'$, we first allocate a *placeholder* state q_X for the variable X. We then build $\mathcal{L}(S')$ in an environment that maps X to q_X. When the construction encounters the variable X, it returns q_X (creating a back-edge). Let $\mathcal{L}(S') = (Q', \Sigma', \delta', q'_\top, q'_\bot)$.

We *merge* the placeholder into the entry state: every transition targeting q_X is redirected to $q'_\top$.

$$\mathcal{L}(S) = (Q' \setminus \{q_X\},\ \ \Sigma',\ \ \delta'[q_X \mapsto q'_\top],\ \ q'_\top,\ \ q'_\bot)$$

The back-edges from X to $q'_\top$ create cycles in the transition graph.

Remark 3.9 (Cycles and SCC quotient)*.* Recursion breaks acyclicity: the state space of $\mu X.\, S$ may contain cycles, and the reachability relation is only a preorder (not a partial order). The remedy is standard: quotient by *strongly connected components* (SCCs). The quotient graph is a DAG, and the reachability ordering on the quotient is a partial order. The following lemma ensures that the quotient preserves boundedness.

Lemma 3.10 (Top Absorption)**.** *Let $(L, \leq)$ be a finite bounded poset with greatest element $\top$, and let $D \subseteq L$ be an* upward-closed *set containing $\top$ (i.e., if $d \in D$ and $d \leq x$, then $x \in D$). Define $L' = (L \setminus D) \cup \{\top\}$ with the restricted ordering. Then L' is a bounded poset with $\bot_{L'} = \bot_L$ and $\top_{L'} = \top$.*

Proof. Since D is upward-closed and $L \setminus D$ is downward-closed (an order-ideal), the bottom element $\bot \notin D$ (otherwise $D = L$, a degenerate case), so L' retains both extrema. The ordering on $L \setminus D$ is inherited from L; the collapsed element $\top$ is above everything in L' by definition.

In the session-type setting, D is the set of SCC-classes that become mutually reachable with the initial state $q_\top$ after adding back-edges from recursion. This set is upward-closed: if $[s] \in D$ and $[s] \leq [r]$ in the quotient, then r reaches s (by $[s] \leq [r]$), s reaches $q_\top$ (since $[s] \in D$), and $q_\top$ reaches r (as initial state); so r and $q_\top$ are mutually reachable, giving $[r] \in D$.

For SCCs that do not contain $q_\top$: guardedness ensures that the SCC consists of states that can still make progress (at least one action before looping), so collapsing the SCC to a representative produces an internal node in the quotient DAG. □

3.6 Rule 6: Parallel Composition

Definition 3.11 (Construction for parallel)**.** Given $S = S_1 \parallel S_2$, let $\mathcal{L}(S_i) = (Q_i, \Sigma_i, \delta_i, q_\top^{S_i}, q_\bot^{S_i})$ for $i = 1, 2$. The *product state space* is:

$$\mathcal{L}(S_1 \parallel S_2) = (Q_1 \times Q_2,\ \ \Sigma_1 \cup \Sigma_2,\ \ \delta,\ \ (q_\top^{S_1}, q_\top^{S_2}),\ \ (q_\bot^{S_1}, q_\bot^{S_2}))$$

where the transition relation allows either component to step independently:

$$\begin{aligned}\delta = \{((s_1, s_2), m, (s_1', s_2)) \mid (s_1, m, s_1') \in \delta_1\} \\ \cup \{((s_1, s_2), m, (s_1, s_2')) \mid (s_2, m, s_2') \in \delta_2\}\end{aligned}$$

Only states reachable from $(q_\top^{S_1}, q_\top^{S_2})$ are included.

Remark 3.12 (Product lattice)*.* The componentwise ordering on $Q_1 \times Q_2$ (where $(s_1, s_2) \leq (t_1, t_2)$ iff $s_1 \leq t_1$ and $s_2 \leq t_2$) is a classical lattice-theoretic construction: the product of two bounded lattices is a bounded lattice [8, 2]. This is the key property that makes parallel composition *force* lattice structure: $\mathcal{L}(S_1 \parallel S_2) = \mathcal{L}(S_1) \times \mathcal{L}(S_2)$. The n-ary case $S_1 \parallel \cdots \parallel S_n$ is obtained by nesting; since the product of bounded lattices is associative [8], the result is independent of bracketing.

3.7 Worked Example

Consider the session type

$$S = \&\{\mathit{open}{:}\&\{\mathit{read}{:}\&\{\mathit{close}{:}\mathbf{end}\},\ \ \mathit{write}{:}\&\{\mathit{ack}{:}\mathbf{end}\}\}\}$$

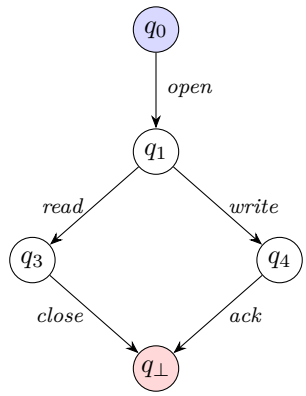
which describes a file protocol: open the file, then either read (and close) or write (and receive an acknowledgement). We build $\mathcal{L}(S)$ bottom-up.

Step 1: Terminal branches. $\mathcal{L}(\mathbf{end}) = (\{q_\bot\}, \emptyset, \emptyset, q_\bot, q_\bot)$.

Step 2: Inner branches. $\mathcal{L}(\&\{\mathit{close}{:}\mathbf{end}\})$: introduce fresh q_3, states $\{q_3, q_\bot\}$, transition $(q_3, \mathit{close}, q_\bot)$. Similarly, $\mathcal{L}(\&\{\mathit{ack}{:}\mathbf{end}\})$: fresh q_4, states $\{q_4, q_\bot\}$, transition $(q_4, \mathit{ack}, q_\bot)$.

Step 3: Outer branch. $S' = \&\{\mathit{read}{:}\&\{\mathit{close}{:}\mathbf{end}\},\ \mathit{write}{:}\&\{\mathit{ack}{:}\mathbf{end}\}\}$. Introduce fresh q_1; share terminal $q_\bot$. States: $\{q_1, q_3, q_4, q_\bot\}$. Transitions: $(q_1, \mathit{read}, q_3)$ and $(q_1, \mathit{write}, q_4)$, plus the internal transitions $(q_3, \mathit{close}, q_\bot)$ and $(q_4, \mathit{ack}, q_\bot)$.

Step 4: Root branch. $S = \&\{open{:}S'\}$. Introduce fresh q_0. Transition: $(q_0, open, q_1)$. Final state space: $q_\top = q_0$, $q_\bot = q_\bot$.



The resulting state space has 5 states, 4 transitions, height 3, and forms a bounded poset with q_0 as top and $q_\bot$ as bottom.

4 Reachability Ordering

The transition structure of a state space induces a natural ordering on states.

Definition 4.1 (Reachability)**.** Let $\mathcal{L}(S) = (Q, \Sigma, \delta, q_\top, q_\bot)$. We define the *reachability relation* $\to^*$ on Q as the reflexive-transitive closure of $\to$:

$$s_1 \to^* s_2 \quad \iff \quad \text{there is a (possibly empty) path from } s_1 \text{ to } s_2 \text{ in } (Q, \delta).$$

That is, $s_1 \to^* s_2$ iff there exist states $s_1 = t_0, t_1, \ldots, t_k = s_2$ and labels $a_1, \ldots, a_k$ such that $t_{j-1} \xrightarrow{a_j} t_j$ for all $1 \le j \le k$. The case $k = 0$ gives $s \to^* s$ (reflexivity). We write $s_1 \ge s_2$ ("s_1 is above s_2") as a synonym for $s_1 \to^* s_2$.

Remark 4.2 (Ordering convention)*.* We adopt the convention that the initial state is the *greatest* element ($q_\top \ge s$ for all s, since $q_\top$ can reach everything) and the terminal state is the *least* ($s \ge q_\bot$ for all s, since everything reaches $q_\bot$). This matches the intuition that "more protocol remaining" corresponds to being higher in the ordering. The notation $\to^*$ follows standard process algebra usage [15, 18]; the ordering $\ge$ follows lattice-theoretic convention [8].

The key property of finite types is that the reachability relation has no non-trivial cycles, making it a partial order.

Proposition 4.3 (Acyclicity)**.** *For every recursion-free session type S, the state space $\mathcal{L}(S)$ is acyclic: there is no sequence of transitions $s = t_0 \xrightarrow{a_1} t_1 \xrightarrow{a_2} \cdots \xrightarrow{a_k} t_k = s$ with $k \ge 1$.*

Proof. By structural induction on S.

Base case: $S = \mathbf{end}$. The state space has a single state and no transitions, so it is trivially acyclic.

Inductive case: branch. $S = \&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$. By the induction hypothesis, each $\mathcal{L}(S_i)$ is acyclic. The construction introduces a fresh state q_0 with only outgoing transitions (to each $q_\top^{S_i}$) and no incoming transitions from any sub-construction. Since q_0 is fresh and has no incoming edges from states in any Q_i, no cycle can pass through q_0. The identification of terminal states $q_\bot^{S_i} \mapsto q_\bot$ is also safe: $q_\bot$ has no outgoing transitions (it is terminal in each sub-construction), so no cycle is created. Since each $\mathcal{L}(S_i)$ is acyclic and the merging of terminal states introduces no new paths, $\mathcal{L}(S)$ is acyclic.

Inductive case: selection. Identical to branch (the graph topology is the same).

The recursion and parallel cases are addressed in the Boundedness section (Section 5): recursion introduces cycles handled by SCC quotient; parallel preserves acyclicity componentwise. □

Proposition 4.4 (Partial order)**.** *For every recursion-free session type S, the reachability relation $\to^*$ on $\mathcal{L}(S)$ is a* partial order*: it is reflexive, antisymmetric, and transitive. For recursive types, $\to^*$ on the SCC quotient $Q^S/\sim$ is a partial order.*

Proof. Reflexivity holds by definition (take the empty path). Transitivity holds by concatenation of paths. Antisymmetry: if $s_1 \to^* s_2$ and $s_2 \to^* s_1$ with $s_1 \neq s_2$, then concatenating the witnessing paths gives a cycle, contradicting Proposition 4.3. □

Definition 4.5 (Reachability poset)**.** The *reachability poset* of a well-formed session type S is $(Q^S, \to^*)$ if S is recursion-free, or $(Q^S/\sim, \to^*)$ (the SCC quotient) if S contains recursion. See also Definition 5.2.

5 Boundedness

We now prove the main result: the reachability poset of every well-formed session type is bounded.

Definition 5.1 (Bounded poset)**.** A poset $(P, \leq)$ is *bounded* if it has a greatest element $\top$ (with $\top \geq p$ for all $p \in P$) and a least element $\bot$ (with $\bot \leq p$ for all $p \in P$).

Definition 5.2 (Canonical poset)**.** For a well-formed session type S, the *canonical poset* $Pos(S)$ is:

- $(Q^S, \to^*)$ if S is recursion-free (the state set under reachability ordering);
- $(Q^S/\sim, \to^*)$ if S contains recursion (the SCC quotient under induced reachability).

In both cases the quotient is the identity when there are no cycles.

Theorem 5.3 (Bounded poset)**.** *For every well-formed session type S, the canonical poset $Pos(S)$ is a finite bounded poset with $\top = [q_\top^S]$ and $\bot = [q_\bot^S]$.*

Proof. We prove by structural induction on S that:

(i) Q^S is finite,
(ii) $q_\top^S \to^* s$ for all $s \in Q^S$ (top reaches everything),
(iii) $s \to^* q_\bot^S$ for all $s \in Q^S$ (everything reaches bottom).

Base case: $S = \mathbf{end}$. $Q = \{q\}$ with $q_\top = q_\bot = q$. Finiteness is trivial. Properties (ii) and (iii) hold by reflexivity.

Inductive case: $S = \&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$. By the induction hypothesis, for each i:

- Q_i is finite,
- $q_\top^{S_i} \to^* s$ for all $s \in Q_i$,
- $s \to^* q_\bot^{S_i}$ for all $s \in Q_i$.

Finiteness: $Q^S = \{q_0\} \cup \bigcup_i Q_i[\, q_\bot^{S_i} \mapsto q_\bot]$ is a finite union of finite sets with one additional element.

Top reaches everything: We need $q_0 \to^* s$ for all $s \in Q^S$. For $s = q_0$, this holds by reflexivity. For $s \in Q_i$, note that $q_0 \xrightarrow{m_i} q_\top^{S_i}$ and $q_\top^{S_i} \to^* s$ by the induction hypothesis, so $q_0 \to^* s$ by transitivity. For $s = q_\bot$ (the shared terminal), pick any i; then $q_0 \xrightarrow{m_i} q_\top^{S_i} \to^* q_\bot^{S_i} = q_\bot$.

Everything reaches bottom: For $s = q_\bot$, reflexivity. For $s = q_0$, pick any i: $q_0 \xrightarrow{m_i} q_\top^{S_i} \to^* q_\bot^{S_i} = q_\bot$. For $s \in Q_i \setminus \{q_\bot^{S_i}\}$, by the induction hypothesis $s \to^* q_\bot^{S_i} = q_\bot$.

Inductive case: $S = \oplus\{l_1{:}S_1, \ldots, l_n{:}S_n\}$. The argument is identical to the branch case, since the graph topology is the same.

Inductive case: $S = \mu X.\, S'$**.** Let $\mathcal{L}(S') = (Q', \Sigma', \delta', q'_\top, q'_\bot)$. The placeholder merge redirects back-edges to $q'_\top$, creating cycles but not adding new states beyond Q'. By the induction hypothesis on S': $q'_\top$ reaches everything in Q', and everything reaches $q'_\bot$. The merge only adds edges *to* $q'_\top$, which cannot break these properties. Thus $q'_\top \to^* s$ for all s and $s \to^* q'_\bot$ for all s.

The cycles break antisymmetry, so the reachability relation on Q' is a preorder, not a partial order. Quotienting by SCCs recovers a partial order on the quotient $Q'/{\sim}$, which is bounded by $[q'_\top]$ and $[q'_\bot]$. The Top Absorption Lemma (Lemma 3.10) guarantees that the quotient preserves bounded poset structure.

Inductive case: $S = S_1 \| S_2$**.** By the induction hypothesis, $\mathcal{L}(S_1)$ and $\mathcal{L}(S_2)$ are finite bounded posets. The product $Q_1 \times Q_2$ with componentwise ordering is finite (since both factors are finite) and bounded with $\top = (q_\top^{S_1}, q_\top^{S_2})$ and $\bot = (q_\bot^{S_1}, q_\bot^{S_2})$. For any $(s_1, s_2) \in Q_1 \times Q_2$: $(q_\top^{S_1}, q_\top^{S_2}) \to^* (s_1, s_2)$ because $q_\top^{S_1} \to^* s_1$ and $q_\top^{S_2} \to^* s_2$; and $(s_1, s_2) \to^* (q_\bot^{S_1}, q_\bot^{S_2})$ because $s_1 \to^* q_\bot^{S_1}$ and $s_2 \to^* q_\bot^{S_2}$. Moreover, the product of two bounded lattices is a bounded lattice [8]. □

Corollary 5.4 (Size bound)**.** *For every recursion-free session type S, $|Q^S| \le |S|$. For recursive types, $|Q^S/{\sim}| \le |S|$ (counting SCC representatives). For parallel, $|Q^{S_1 \| S_2}| = |Q^{S_1}| \cdot |Q^{S_2}|$.*

Proof. The recursion-free case is by induction. For **end**: $|Q| = 1 = |\mathbf{end}|$. For $\&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$: $|Q^S| = 1 + \sum_i |Q^{S_i}| - (n-1) \le 1 + \sum_i |S_i| = |S|$, where the $-(n-1)$ accounts for the n terminal states being identified into one. The selection case is identical. For recursion: the placeholder-merge does not add states beyond the body, and SCC quotient only reduces state count. For parallel: the product construction is exact. □

Corollary 5.5 (Acyclicity)**.** *For every recursion-free session type S, $\mathcal{L}(S)$ is a directed acyclic graph (DAG). For recursive types, the SCC quotient $Q^S/{\sim}$ is a DAG.*

Proposition 5.6 (Complexity)**.** *The state-space construction $\mathcal{L}(S)$ runs in time $O(|S|)$ for recursion-free types without parallel, and $O(|S| + |Q^{S_1}| \cdot |Q^{S_2}|)$ for types containing $S_1 \| S_2$. The SCC quotient adds $O(|Q| + |\delta|)$ (Tarjan's algorithm [20]). The bounded poset check (verifying reachability from $q_\top$ and to $q_\bot$) is $O(|Q| + |\delta|)$ via two graph traversals.*

Proof. Each construction rule introduces at most one fresh state and performs a constant number of operations per sub-construction, giving a linear traversal of the AST. The parallel case is the exception: the product construction creates $|Q_1| \times |Q_2|$ states and up to $|Q_1| \cdot |\delta_2| + |Q_2| \cdot |\delta_1|$ transitions. SCC detection uses Tarjan's algorithm [20] in linear time. Reachability checks use BFS/DFS from $q_\top$ (forward) and $q_\bot$ (backward). □

Remark 5.7 (Space complexity). The state space requires $O(|Q| + |\delta|)$ space. For types with k nested parallel compositions, the worst case is $O(\prod_{i=1}^{k} |Q_i|)$ states (exponential in k), but in practice the sub-state-spaces of parallel branches are small (typically 2–5 states each). Across all 79 benchmarks, the largest state space has 41 states (Ki3 CI/CD Pipeline, with 78 transitions).

6 Properties of the Construction

The state-space construction enjoys several structural properties that will be important in later work.

Proposition 6.1 (Determinism)**.** *For every well-formed session type S and every state $s \in Q^S$, if $s \xrightarrow{m} s'$ and $s \xrightarrow{m} s''$, then $s' = s''$. That is, $\mathcal{L}(S)$ is* deterministic*: each label determines at most one successor.*

Proof. By induction on S. For **end**: no transitions, vacuously true. For branch/selection: the labels are pairwise distinct (Definition 2.1), so transitions from q_0 are deterministic; sub-constructions are deterministic by hypothesis and have disjoint state sets (except the shared $q_\perp$, which has no outgoing transitions). For recursion $\mu X.\, S'$: the placeholder-merge redirects back-edges to $q'_\top$ but does not add new labels at any state; determinism of $\mathcal{L}(S')$ is preserved. For parallel $S_1 \parallel S_2$: at any product state (s_1, s_2), a label m belongs to at most one component (label sets of S_1 and S_2 are disjoint at each state by the induction hypothesis), so the transition is uniquely determined. □

Proposition 6.2 (Label disjointness)**.** *In the construction for* $\&\{m_1{:}S_1, \ldots, m_n{:}S_n\}$ *(resp. selection), the labels of the outgoing transitions from the initial state are pairwise disjoint. More precisely,* $m_i \neq m_j$ *for* $i \neq j$*.*

Proof. Immediate from the well-formedness condition in Definition 2.1 requiring pairwise distinct labels. □

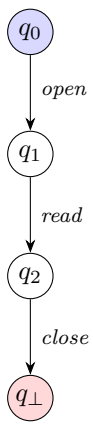
Proposition 6.3 (Compositionality)**.** *The construction* $\mathcal{L}(-)$ *is* compositional*:* $\mathcal{L}(S)$ *depends only on* S *and the constructions* $\mathcal{L}(S_1), \ldots, \mathcal{L}(S_n)$ *of its immediate sub-terms. In particular, replacing a sub-term* S_i *with* S'_i *such that* $\mathcal{L}(S_i) = \mathcal{L}(S'_i)$ *(up to state renaming) does not change* $\mathcal{L}(S)$ *(up to state renaming).*

Proof. By inspection of all six construction rules: each rule combines sub-constructions with fresh states and transitions, without inspecting their internal structure beyond top state, bottom state, and transition relation. Recursion (Section 3.5) adds back-edges to the entry state of the body, depending only on the body's top state. Parallel (Section 3.6) forms the product, depending only on each component's state set and transitions. □

7 Examples

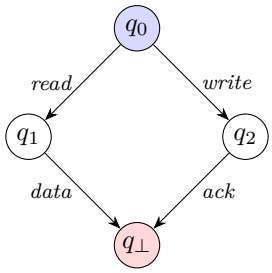
We illustrate the construction with six examples of increasing complexity, covering the finite fragment, recursion, and parallel composition.

Example 7.1 (Simple chain). $S = \&\{\mathit{open}{:}\&\{\mathit{read}{:}\&\{\mathit{close}{:}\mathbf{end}\}\}\}$. The state space is a 4-element chain:



$|Q| = 4$, $|\delta| = 3$, height $= 3$. A total order: every pair of states is comparable.

Example 7.2 (Diamond (branch)). $S = \&\{\mathit{read}{:}\&\{\mathit{data}{:}\mathbf{end}\},\ \ \mathit{write}{:}\&\{\mathit{ack}{:}\mathbf{end}\}\}$. The state space forms a diamond (M_2 lattice):

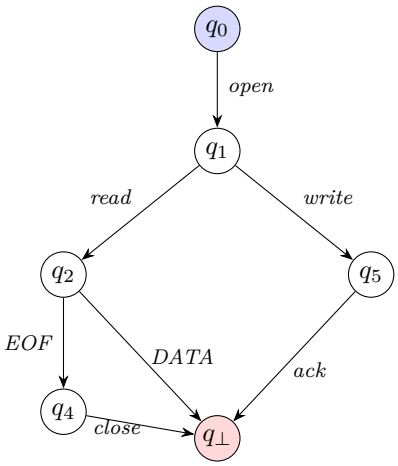


$|Q| = 4$, $|\delta| = 4$, height $= 2$. The states q_1 and q_2 are incomparable: $q_1 \not\to^* q_2$ and $q_2 \not\to^* q_1$. This is the simplest non-chain state space. Note that this is already a lattice: $q_0 = q_1 \vee q_2$ and $q_\perp = q_1 \wedge q_2$.

Example 7.3 (Nested branch with selection).

$$S = \&\{\mathit{open}{:}\&\{\mathit{read}{:}\oplus\{\mathit{DATA}{:}\mathbf{end},\ \mathit{EOF}{:}\&\{\mathit{close}{:}\mathbf{end}\}\},\ \mathit{write}{:}\&\{\mathit{ack}{:}\mathbf{end}\}\}\}$$

This describes a file protocol: open the file, then either read (receiving either data or end-of-file, with a close operation after EOF) or write (receiving an acknowledgement).

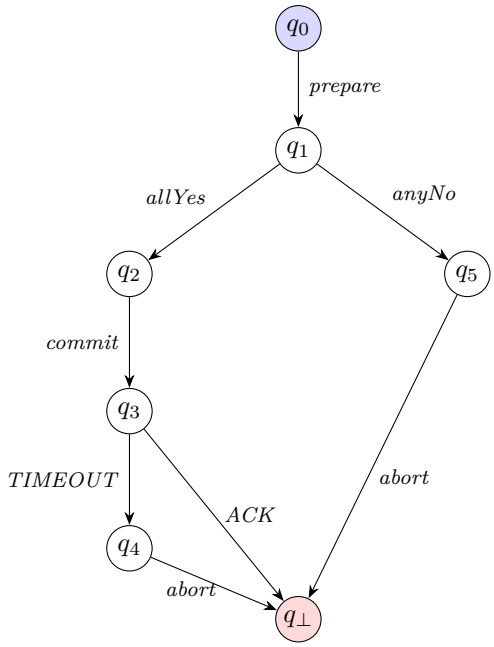


$|Q| = 6$, $|\delta| = 6$, height $= 4$. The selection at q_2 (transitions DATA, EOF) is an internal choice: the object decides whether there is more data.

Example 7.4 (Two-Phase Commit). From the benchmark suite:

$$S = \&\{\mathit{prepare}{:}\&\{\mathit{allYes}{:}\&\{\mathit{commit}{:}\oplus\{\mathit{ACK}{:}\mathbf{end},\ \mathit{TIMEOUT}{:}\&\{\mathit{abort}{:}\mathbf{end}\}\}\},\ \mathit{anyNo}{:}\&\{\mathit{abort}{:}\mathbf{end}\}\}\}$$

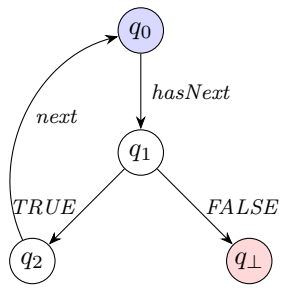
This describes a simplified two-phase commit coordinator: prepare the transaction, then either all participants vote yes (commit, with the outcome being either an ACK or a timeout requiring abort) or some vote no (abort directly).



$|Q| = 7$, $|\delta| = 8$, height $= 5$. This state space is a bounded poset that also happens to be a lattice.

Example 7.5 (Recursive iterator). $S = \mu X.\, \&\{hasNext\colon \oplus\{TRUE\colon \&\{next\colon X\},\ \ FALSE\colon \mathbf{end}\}\}$.

This is the Java Iterator protocol: call *hasNext*, the object decides TRUE or FALSE; if TRUE, call *next* and repeat. The variable X creates a back-edge from the post-*next* state to the entry state, forming a cycle.



The back-edge $q_2 \xrightarrow{next} q_0$ creates a cycle $q_0 \to q_1 \to q_2 \to q_0$. The SCC quotient collapses $\{q_0, q_1, q_2\}$ into a single representative $[q_0]$, yielding a 2-element bounded poset $[q_0] \to^* [q_\perp]$.

Example 7.6 (Parallel composition). $S = \&\{read\colon \mathbf{end}\} \parallel \&\{write\colon \mathbf{end}\}$.

Two operations execute concurrently: a read path and a write path. The product construction yields a $2 \times 2 = 4$-state lattice:

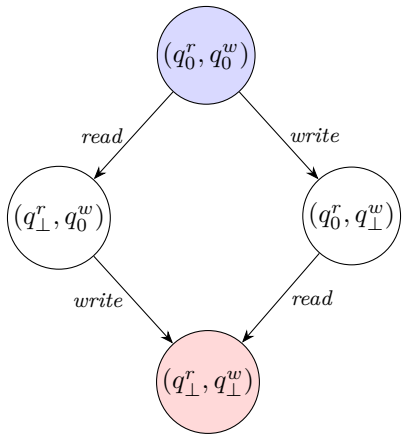


Table 1: State-space metrics for the six worked examples.

Protocol	Constructors	$\|Q\|$	$\|\delta\|$	SCCs	Notes
Simple chain	&	4	3	4	total order, Example 7.1
Diamond	&	4	4	4	M_2 lattice, Example 7.2
File object	&	5	4	5	Section 3.7
Nested selection	&, $\oplus$	6	6	6	Example 7.3
Two-Phase Commit	&, $\oplus$	7	8	7	Example 7.4
Java Iterator	&, $\oplus$, μ	4	4	2	SCC quotient, Example 7.5
Read $\parallel$ Write	$\parallel$	4	4	4	product, Example 7.6

The four states form a diamond lattice (isomorphic to M_2). The componentwise ordering makes $(q_\perp^r, q_0^w)$ and $(q_0^r, q_\perp^w)$ incomparable—they represent "read done, write pending" and "read pending, write done" respectively. Both operations must complete before reaching $\perp$. This is the simplest instance of the product lattice.

Table 1 summarises the six worked examples; Table 2 presents state-space metrics for a representative selection of 40 protocols from our 79-benchmark suite, grouped by domain.

8 Discussion and Future Work

Is it always a lattice? We have shown that $\mathcal{L}(S)$ is a bounded poset. A natural question is whether it is always a *lattice*—whether every pair of states has a meet and a join. The bounded poset property is the essential ingredient: once we know every state space has unique top and bottom with full reachability, the lattice proof reduces to showing pairwise meets and joins exist. For branch/selection, these arise from the shared top and bottom of distinct sub-state-spaces (the initial state is the join, the terminal state the meet of incomparable states in different branches). For parallel, meets and joins are componentwise (product of lattices). For recursion, the Top Absorption Lemma (Lemma 3.10) preserves meets, and joins that land in the collapsed set become $\top$. The full lattice theorem and its proof are the subject of forthcoming work.

Empirical validation. We have verified the lattice property on all 79 benchmark protocols, spanning 13 application domains: networking (SMTP, HTTP, TLS, WebSocket, MQTT, DNS, gRPC), distributed systems (Two-Phase Commit, Raft, Saga, Failover, Leader Replication), databases (JDBC, Connection Pool), AI agents (MCP, A2A), security (OAuth 2.0), programming patterns (Java Iterator, File Object, Reentrant Lock, Rate Limiter, Circuit Breaker, Pub/Sub), tool pipelines (Reticulate Pipeline, GitHub CI), biological processes (enzyme kinetics, ion channels), physics (Carnot Cycle, quantum measurement, billiard map), commerce (Two-Buyer, ATM), and industrial SaaS provisioning (Ki3 Onboarding, Ki3 Offboarding, Ki3 CI/CD Pipeline). An exhaustive enumeration of 478,000+ terminating session types at depth 3 found zero counterexamples. Counterexamples arise only from non-terminating recursive types which violate well-formedness.

Remark 8.1 (Towards the full lattice theorem). The bounded poset property (Theorem 5.3) is the base case for proving the full lattice theorem by structural induction. Preliminary results indicate that 80% of benchmarks produce distributive lattices; the remaining 20% contain the pentagon N_5 as a forbidden sublattice. The key lemmas are mechanised in Lean 4 (28 modules, 4,150 lines, zero `sorry`).

Tool support. The construction is implemented in two tools: Reticulate (Python, 2,428 tests) and Bica Reborn (Java, 1,619 tests). Both implement the same pipeline:

$$\text{session type string} \xrightarrow{\text{parse}} \text{AST} \xrightarrow{\text{build}} \mathcal{L}(S) \xrightarrow{\text{check}} \text{lattice?} \xrightarrow{\text{render}} \text{Hasse diagram}$$

Table 2: State-space metrics for 40 representative protocols from the 79-benchmark suite. $|Q|$ = states, $|\delta|$ = transitions, SCCs = strongly connected components after quotient, $\|$ = uses parallel (f&f = fire-and-forget, f/j = fork-join). All 79 are confirmed to form bounded lattices.

#	Protocol	$\|Q\|$	$\|\delta\|$	SCCs	$\|$	Domain
1	Java Iterator	4	4	2		Pattern
2	File Object	5	5	4		I/O
3	SMTP	7	8	4		Network
4	HTTP Connection	5	7	3		Network
5	OAuth 2.0	7	11	6		Security
6	Two-Buyer	10	14	10	f&f	Commerce
7	MCP	7	17	5	f&f	AI Agent
8	A2A	5	7	4		AI Agent
9	File Channel	7	12	7	f/j	I/O
10	ATM	7	11	6		Finance
11	Reentrant Lock	6	7	2		Concurrency
12	WebSocket	6	15	6	f&f	Network
13	DB Transaction	5	9	3		Database
14	TLS Handshake	13	15	13		Security
15	Raft Leader	6	11	2		Distributed
16	MQTT Client	8	20	6	f/j	IoT
17	Circuit Breaker	6	10	2		Resilience
18	Connection Pool	13	29	5	f&f	Database
19	Kafka Consumer	11	23	5	f&f	Streaming
20	Saga Orchestrator	20	37	20	f/j	Distributed
21	Two-Phase Commit	7	8	7		Distributed
22	Failover	6	8	3		Resilience
23	Enzyme (M-M)	4	5	2		Biology
24	Ion Channel (Na^+/K^+)	11	19	2	f/j	Biology
25	Ribosome Translation	5	5	3		Biology
26	Cell Cycle	9	12	2		Biology
27	Action Potential	13	21	2	f/j	Biology
28	Carnot Cycle	5	5	2		Physics
29	Quantum Measurement	9	11	9		Physics
30	Double Slit	6	6	6	f/j	Physics
31	Quantum Teleportation	14	40	14	f/j	Physics
32	Needham-Schroeder	5	5	5		Crypto
33	Kerberos	9	14	8		Crypto
34	SSH Handshake	18	25	12		Crypto
35	Mutual TLS	13	24	13	f/j	Crypto
36	Ki3 Onboarding	32	50	32	f/j	SaaS
37	Ki3 Offboarding	16	18	16	f/j	SaaS
38	Ki3 CI/CD Pipeline	41	78	41	f/j	CI/CD
39	GitHub CI Workflow	11	13	11	f/j	CI/CD
40	Reticulate Pipeline	14	18	14	f/j	Tooling

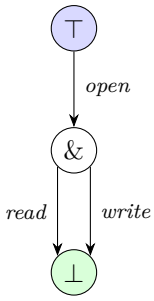


Figure 1: Hasse diagram generated by RETICULATE for the file protocol $\&\{open{:}\&\{read{:}\mathbf{end},\ write{:}\mathbf{end}\}\}$. Blue = initial state ($\top$), green = terminal state ($\bot$). The tool labels nodes by constructor type rather than state identifier.

The *parse* stage tokenises and parses the session type string into an abstract syntax tree. The *build* stage applies the rules of Section 3 by structural induction, producing the state space $\mathcal{L}(S)$. The *check* stage computes the SCC quotient (if recursive), verifies boundedness, and tests the lattice property by checking all pairwise meets and joins. The *render* stage generates a Hasse diagram in DOT format (Graphviz) or SVG. Both tools are open-source and available as prebuilt binaries (Python wheel and Java JAR respectively). For example, running RETICULATE on a simplified file protocol (without the inner *close*/*ack* steps of Section 3.7):

```
$ reticulate '&{open: &{read: end, write: end}}'
Session type: &{open: &{read: end, write: end}}
States: 3  |  Transitions: 3  |  SCCs: 3

Lattice check: IS a lattice
  Top reachable:    yes
  Bottom reachable: yes
  All meets exist:  yes
  All joins exist:  yes
```

The `--dot` flag outputs a Hasse diagram in Graphviz DOT format. Figure 1 shows the rendered diagram for this example. BICA REBORN produces identical output from `java -jar bica-reborn.jar`.
The tools, 79 binary benchmarks, and Lean 4 proofs are archived as supplementary material [4] and will be made available on a permanent repository upon publication.

Lean 4 mechanisation. The key lemmas (End, Branch/Choice, Selection, Recursion with Top Absorption, Parallel) are mechanised in Lean 4 [9] with zero `sorry`, totalling 890 lines. The assembly of these lemmas into the main theorem (`ReticulateTheorem.lean`) confirms that the induction closes.

Related work. Session types originate with Honda [14] for the π-calculus and were adapted to objects by Gay, Vasconcelos, and Ravara [22, 13]. The BICA system [12] introduced session types as annotations for Java objects, and MUNGO [17] extended this with typechecking integrated into Java's type system. Multiparty session types [15] and the Scribble protocol language [24] generalize from binary to n-party protocols.

The state-space perspective on session types is implicit in all of this work—the "typestate" view of Strom–Yemini [19] and DeLine–Fähndrich [10] makes it explicit for object-oriented languages. Kobayashi [16] constructs *type graphs* for process types that are structurally similar to our state spaces, though without the order-theoretic analysis. Dardha and Gay [6] use a linear logic interpretation that implicitly constructs derivation trees with bounded structure.

Scalas and Yoshida [18] develop a "less is more" approach to session-type checking that builds LTSs from types—the closest precursor to our construction, though they do not study lattice properties. Vasconcelos [21] provides a comprehensive foundation for session types. Dezani-Ciancaglini et al. [11] develop session types for object-oriented languages. Dardha, Giachino, and Sangiorgi [7] revisit session types through linear logic. Caires and Pfenning [3] establish a Curry–Howard correspondence between session types and linear logic, where the connectives & and $\oplus$ correspond to our branch and selection; Wadler [23] gives a classical account. Ancona et al. [1] survey behavioral type formalisms; the lattice structure we identify could potentially extend beyond session types to other behavioral type systems such as typestate.

The *order-theoretic* properties of session-type state spaces (bounded poset, lattice) appear to be novel. Lattice theory in programming languages is most associated with abstract interpretation [5] and domain theory [8]; our use of lattices for *protocol structure* rather than *abstract values* is, to our knowledge, new.

9 Conclusion

We have defined the state-space construction $\mathcal{L}(S)$ for binary session types by structural induction, covering all six constructors: termination, branch, selection, recursion, parallel, and variables. We proved that the reachability relation on $\mathcal{L}(S)$ is a bounded poset for the finite fragment, and extends to a bounded poset on the SCC quotient when recursion introduces cycles. Parallel composition produces a product state space, inheriting bounded lattice structure from the classical result on products of lattices. We established determinism and compositionality, and verified the construction on 79 benchmark protocols.

The bounded poset property is the base case for the full lattice theorem (that session-type state spaces are lattices), with consequences for subtyping, bisimulation, and concurrent access patterns.

The construction is implemented in the Reticulate library (Python, 2,428 tests) and in Bica Reborn (Java, 1,619 tests), mechanised in Lean 4 (890 lines, zero `sorry`), and validated on 79 benchmark protocols drawn from networking, databases, AI agents, fault tolerance, and biology.

Acknowledgements. The author thanks Vasco T. Vasconcelos for supervision and guidance.

References

[1] Davide Ancona et al. Behavioral types in programming languages. In *Foundations and Trends in Programming Languages*, volume 3, pages 95–230. Now Publishers, 2016.

[2] Garrett Birkhoff. *Lattice Theory*, volume 25 of *AMS Colloquium Publications*. American Mathematical Society, 3rd edition, 1967.

[3] Luís Caires and Frank Pfenning. Session types as intuitionistic linear propositions. In *CONCUR 2010*, volume 6269 of *LNCS*, pages 222–236. Springer, 2010.

[4] Alexandre Zua Caldeira. Binary session types benchmark dataset: 79 protocols with lattice-structured state spaces, 2026.

[5] Patrick Cousot and Radhia Cousot. Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints. *Conference Record of the Fourth ACM Symposium on Principles of Programming Languages*, pages 238–252, 1977.

[6] Ornela Dardha and Simon J. Gay. A new linear logic for deadlock-free session-typed processes. In *FoSSaCS 2018*, volume 10803 of *LNCS*, pages 91–109. Springer, 2018.

[7] Ornela Dardha, Elena Giachino, and Davide Sangiorgi. Session types revisited. *Information and Computation*, 256:253–286, 2017.

[8] B. A. Davey and H. A. Priestley. *Introduction to Lattices and Order*. Cambridge University Press, 2nd edition, 2002.

[9] Leonardo de Moura and Sebastian Ullrich. The Lean 4 theorem prover and programming language. In *CADE 2021*, volume 12699 of *LNCS*, pages 625–635. Springer, 2021.

[10] Robert DeLine and Manuel Fähndrich. Typestates for objects. In *ECOOP 2004*, volume 3086 of *LNCS*, pages 465–490. Springer, 2004.

[11] Mariangiola Dezani-Ciancaglini, Dimitris Mostrous, Nobuko Yoshida, and Sophia Drossopoulou. Session types for object-oriented languages. In *ECOOP 2006*, pages 328–352. Springer, 2006.

[12] Simon Gay, Vasco T. Vasconcelos, António Ravara, Nils Gesbert, and Alexandre Zua Caldeira. Modular session types for distributed object-oriented programming. In *POPL 2010*, pages 299–312. ACM, 2010.

[13] Simon J. Gay and Malcolm Hole. Subtyping for session types in the pi calculus. *Acta Informatica*, 42(2–3):191–225, 2005.

[14] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. Language primitives and type discipline for structured communication-based programming. In *ESOP 1998*, volume 1381 of *LNCS*, pages 122–138. Springer, 1998.

[15] Kohei Honda, Nobuko Yoshida, and Marco Carbone. Multiparty asynchronous session types. In *POPL 2008*, pages 273–284. ACM, 2008.

[16] Naoki Kobayashi. A new type system for deadlock-free processes. *Information and Computation*, 204(6):840–911, 2006.

[17] Dimitrios Kouzapas, Ornela Dardha, Roly Perera, and Simon J. Gay. Typechecking protocols with Mungo and StMungo. In *PPDP 2016*, pages 146–159. ACM, 2016.

[18] Alceste Scalas and Nobuko Yoshida. Less is more: Multiparty session types revisited. In *POPL 2019*, pages 37:1–37:29. ACM, 2019.

[19] Robert E. Strom and Shaula Yemini. Typestate: A programming language concept for enhancing software reliability. *IEEE Transactions on Software Engineering*, SE-12(1):157–171, 1986.

[20] Robert Tarjan. Depth-first search and linear graph algorithms. *SIAM Journal on Computing*, 1(2):146–160, 1972.

[21] Vasco T. Vasconcelos. Fundamentals of session types. *Information and Computation*, 217:52–70, 2012.

[22] Vasco T. Vasconcelos, Simon J. Gay, and António Ravara. Type checking a multithreaded functional language with session types. *Theoretical Computer Science*, 368(1–2):64–87, 2006.

[23] Philip Wadler. Propositions as sessions. In *ICFP 2012*, pages 273–286. ACM, 2012.

[24] Nobuko Yoshida, Raymond Hu, Rumyana Neykova, and Nicholas Ng. The Scribble protocol language. In *TGC 2013*, volume 8358 of *LNCS*, pages 22–41. Springer, 2013.